

■ Deep Learning to Modern LLMs: The Complete Masterclass

Comprehensive Course Guide & Notebook: Every Mathematical Foundation, Model Architecture, Code Implementation, and Production Step Explained

Scope: Lessons 1–32 (NumPy Foundations → Micrograd Autograd → PyTorch → WaveNet → Transformers → BERT → GPT-2 → Phi-3.5 → LoRA → QLoRA → GGUF → Ollama)

Chapter 1: Neural Networks & Autograd from Scratch (Lessons 1–4)

Before modern frameworks, deep learning is governed by linear algebra and recursive calculus. Every neural network transforms input $\mathbf{x} \in \mathbb{R}^{d_{in}}$ into predictions $\hat{\mathbf{y}} \in \mathbb{R}^{d_{out}}$ via affine mappings followed by non-linear activations:

1. Mathematical Formulation of Multi-Class Perceptron:

- **Affine Transformation:** $\mathbf{z} = \mathbf{W} \mathbf{x} + \mathbf{b}$ where $\mathbf{W} \in \mathbb{R}^{C \times D}$, $\mathbf{b} \in \mathbb{R}^C$.
- **Softmax Probability Distribution:** $P(Y=k | \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^C e^{z_j}}$ (normalizes logits into valid probabilities $\sum p_k = 1$).
- **Cross-Entropy Loss Objective:** $\mathcal{L} = -\sum_{k=1}^C y_k \log(\hat{y}_k)$ where $\mathbf{y}$ is the one-hot target vector.
- **Gradient of Loss w.r.t Logits:** $\frac{\partial \mathcal{L}}{\partial z_k} = \hat{y}_k - y_k$ (the prediction error directly drives the weight update!).

```
# Lesson 1: NumPy Softmax & Cross-Entropy from Scratch
def softmax(logits):
    exp_shifted = np.exp(logits - np.max(logits, axis=1, keepdims=True)) # Numerical stability
    return exp_shifted / np.sum(exp_shifted, axis=1, keepdims=True)

def compute_loss_and_grad(W, b, X, y_onehot):
    N = X.shape[0]
    probs = softmax(X @ W + b)
    loss = -np.sum(y_onehot * np.log(probs + 1e-12)) / N
    dlogits = (probs - y_onehot) / N
    dW = X.T @ dlogits
    db = np.sum(dlogits, axis=0, keepdims=True)
    return loss, dW, db
```

2. Micrograd: Reverse-Mode Automatic Differentiation Engine:

Every operation builds a directed acyclic graph (DAG). The chain rule $\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial x}$ is computed in topological order:

```
# Lesson 2: Scalar Autograd Engine (Value Class)
class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data, self.grad = float(data), 0.0
        self._backward = lambda: None
        self._prev, self._op = set(_children), _op

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), '+')
        def _backward():
            self.grad += 1.0 * out.grad; other.grad += 1.0 * out.grad
        out._backward = _backward
        return out

    def backward(self): # Topological sort backward execution
        topo, visited = [], set()
        def build(v):
            if v not in visited:
                visited.add(v)
                for child in v._prev: build(child)
                topo.append(v)
            build(self); self.grad = 1.0
        for v in reversed(topo): v._backward()
```

Chapter 2: PyTorch Core, nn.Module & Training Pipelines (Lessons 5–8)

PyTorch abstracts tensor operations onto GPUs/MPS and organizes neural networks into modular classes subclassing `torch.nn.Module`:

```
# Lessons 5-8: PyTorch Multi-Layer Perceptron Pipeline
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

class CustomMLP(nn.Module):
    def __init__(self, in_features, hidden_dim, out_classes, dropout_p=0.2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, hidden_dim),
            nn.BatchNorm1d(hidden_dim),      # Reduces internal covariate shift
            nn.ReLU(),
            nn.Dropout(dropout_p),          # Regularization by randomly zeroing units
            nn.Linear(hidden_dim, out_classes)
        )
    def forward(self, x):
        return self.net(x)

# Standard Training Loop
def train_epoch(model, dataloader, criterion, optimizer, device):
    model.train()
    total_loss = 0.0
    for x_batch, y_batch in dataloader:
        x_batch, y_batch = x_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()              # Clear accumulated gradients
        outputs = model(x_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()                     # Autograd backpropagation
        optimizer.step()                    # Update weights via Adam/SGD
        total_loss += loss.item() * x_batch.size(0)
    return total_loss / len(dataloader.dataset)
```

Chapter 3: Sequence Modeling: N-Grams, WaveNet & RNNs (Lessons 9–14)

Modeling text requires capturing sequential dependencies. We progress from count-based statistical n-grams to learned continuous representations, dilated causal convolutions, and gated recurrent units:

- **Character Bigrams (Lesson 9):** Computes transition matrix $C[i, j]$ counting how often char j follows char i . Sampling uses multinomial sampling on smoothed row probabilities.
- **Bengio et al. MLP (Lesson 10):** Replaces sparse one-hot encodings with a learned continuous embedding matrix $C \in \mathbb{R}^{|\mathcal{V}| \times d}$. Concatenates k context word embeddings into a dense hidden layer.
- **WaveNet (Lesson 12):** Uses **Dilated Causal Convolutions** where the dilation rate doubles exponentially ($d=1, 2, 4, 8, \dots$), expanding the receptive field exponentially without losing temporal resolution.

```
# Lesson 12 & 14: Dilated Conv & LSTM Cell Mechanics
# 1. WaveNet Dilated Causal Convolution Block:
class WaveNetBlock(nn.Module):
    def __init__(self, channels, dilation):
        super().__init__()
        self.conv = nn.Conv1d(channels, channels * 2, kernel_size=2, dilation=dilation, padding=dilation)
        self.res_out = nn.Conv1d(channels, channels, kernel_size=1)
    def forward(self, x):
        out = self.conv(x)[: :, :-self.conv.padding[0]] # Causal slice
        f, g = torch.chunk(out, 2, dim=1)
        gate = torch.tanh(f) * torch.sigmoid(g)          # Gated Activation Unit
        return x + self.res_out(gate)                     # Residual connection

# 2. LSTM Gated Cell Equations:
# f_t = sigma(W_f x_t + U_f h_{t-1} + b_f) [Forget Gate: What to discard from memory]
# i_t = sigma(W_i x_t + U_i h_{t-1} + b_i) [Input Gate: What new information to store]
# C_t = f_t * C_{t-1} + i_t * tanh(W_c x_t) [Cell State: Uninterrupted gradient highway]
# o_t = sigma(W_o x_t + U_o h_{t-1} + b_o) [Output Gate: Filtered hidden state]
```

Chapter 4: Scaled Dot-Product Attention & GPT from Scratch (Lessons 15–22)

Transformers replace recurrence entirely with the **Scaled Dot-Product Self-Attention Mechanism**, allowing every token to attend to all other tokens in parallel:

ATTENTION MECHANISM & CAUSAL MASK

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}} + M\right) \cdot V$$

Where M is the Causal Attention Mask:

```
[ 0, -inf, -inf, -inf ]
M = [ 0,  0, -inf, -inf ] <-- Forces position t to attend ONLY
     [ 0,  0,  0, -inf ]   to past positions <= t
     [ 0,  0,  0,  0  ]
```

Lessons 16-19: Multi-Head Self-Attention and Full GPT Block

```
class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads, block_size):
        super().__init__()
        self.n_heads, self.head_dim = n_heads, d_model // n_heads
        self.qkv_proj = nn.Linear(d_model, 3 * d_model)
        self.out_proj = nn.Linear(d_model, d_model)
        self.register_buffer('mask', torch.tril(torch.ones(block_size, block_size))
                             .view(1, 1, block_size, block_size))

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.qkv_proj(x).chunk(3, dim=-1)
        q = q.view(B, T, self.n_heads, self.head_dim).transpose(1, 2) # (B, H, T, D)
        k = k.view(B, T, self.n_heads, self.head_dim).transpose(1, 2)
        v = v.view(B, T, self.n_heads, self.head_dim).transpose(1, 2)

        # Scaled Dot Product with Causal Mask
        scores = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim)
        scores = scores.masked_fill(self.mask[:, :, :T, :T] == 0, float('-inf'))
        weights = F.softmax(scores, dim=-1)
        out = weights @ v
        out = out.transpose(1, 2).contiguous().view(B, T, C)
        return self.out_proj(out)

class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, block_size, mlp_ratio=4):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model) # Pre-LN architecture
        self.attn = CausalSelfAttention(d_model, n_heads, block_size)
        self.ln2 = nn.LayerNorm(d_model)
        self.mlp = nn.Sequential(
            nn.Linear(d_model, d_model * mlp_ratio),
            nn.GELU(),
            nn.Linear(d_model * mlp_ratio, d_model)
        )

    def forward(self, x):
        x = x + self.attn(self.ln1(x)) # Residual connection 1
        x = x + self.mlp(self.ln2(x)) # Residual connection 2
        return x
```

Chapter 5: Hugging Face Ecosystem & BERT Fine-Tuning (Lessons 23 & 25)

BERT (Bidirectional Encoder Representations from Transformers) learns deep bidirectional representations. For classification, the first token '[CLS]' aggregates sentence-level context and is projected into class probabilities:

```
# Lesson 25: BERT Classification Pipeline with Dynamic Batch Padding
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          DataCollatorWithPadding, Trainer, TrainingArguments)

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model = AutoModelForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=5)

# Dynamic batch padding collator: Pads ONLY to longest item in batch (saves 40-60% VRAM!)
collator = DataCollatorWithPadding(tokenizer=tokenizer)

training_args = TrainingArguments(
    output_dir='./bert-classifier',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    learning_rate=2e-5,          # Low learning rate preserves pretrained weights
    weight_decay=0.01,           # L2 weight regularization
    warmup_ratio=0.1,            # Linear learning rate warmup over initial 10% steps
    eval_strategy='epoch',
    save_strategy='epoch',
    fp16=torch.cuda.is_available(),
    report_to='none'
)
trainer = Trainer(model=model, args=training_args, train_dataset=ds_train, eval_dataset=ds_val, data_collator=collator)
trainer.train()
```

Chapter 6: GPT-2 Instruction Tuning & Label Loss Masking (Lesson 26)

In instruction tuning, the model must **never** be penalized for predicting the prompt tokens; loss must be computed exclusively on the target assistant completion. This is achieved using PyTorch's `ignore_index=-100`:

Crucial Target Masking Principle: Mathematical Reason for -100: PyTorch's `nn.CrossEntropyLoss(ignore_index=-100)` unconditionally zeroes out the loss contribution for any index with label -100. No gradient flows back from prompt tokens, so the model learns purely to formulate responses given prompts.

```
# Lesson 26: Custom Delimiters, Token Resizing & Target Masking
USER_TOKEN, ASSISTANT_TOKEN = '<|user|>', '<|assistant|>'
tokenizer.add_special_tokens({
    'pad_token': tokenizer.eos_token,
    'additional_special_tokens': [USER_TOKEN, ASSISTANT_TOKEN]
})
# MUST resize embedding matrix so vocabulary size matches model weights!
model.resize_token_embeddings(len(tokenizer))

def tokenize_and_mask_labels(example, max_len=512):
    encoding = tokenizer(example['text'], truncation=True, max_length=max_len, padding='max_length')
    input_ids = encoding['input_ids']
    attention_mask = encoding['attention_mask']
    labels = [-100] * len(input_ids) # Initialize all labels to -100 (ignored)

    assistant_id = tokenizer.convert_tokens_to_ids(ASSISTANT_TOKEN)
    if assistant_id in input_ids:
        start_idx = input_ids.index(assistant_id) + 1
        for i in range(start_idx, len(input_ids)):
            if attention_mask[i] == 1:
                labels[i] = input_ids[i] # Compute loss ONLY on real assistant tokens!
    return {'input_ids': input_ids, 'attention_mask': attention_mask, 'labels': labels}
```

Chapter 7: Modern Chat LLMs & Jinja Chat Templates (Lesson 27)

Modern LLMs (e.g., Microsoft Phi-3.5-mini-instruct, Llama-3) standardize multi-turn conversational dialogue using Jinja2 Chat Templates stored directly in `tokenizer_config.json`:

```
# Lesson 27: Applying Native Chat Templates & Seq2Seq Dynamic Collation
from transformers import DataCollatorForSeq2Seq

messages = [
    {'role': 'system', 'content': 'You are an expert diagnostic assistant.'},
    {'role': 'user', 'content': 'What causes chlorosis in citrus leaves?'}
]
# Automatically renders exact special tokens: <|system|>...<|end|><|user|>...<|end|><|assistant|>
prompt = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)

# DataCollatorForSeq2Seq dynamically pads batch AND replaces label padding with -100:
data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=model, padding='longest')
```

Chapter 8: Parameter-Efficient Fine-Tuning (PEFT) & LoRA Math (Lesson 28)

Full fine-tuning of multi-billion parameter models is computationally prohibitive. **LoRA (Low-Rank Adaptation)** freezes the pretrained base weight $W_0 \in \mathbb{R}^{d \times k}$ and factorizes updates into two low-rank matrices $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ where $r \ll \min(d, k)$:

LORA MATHEMATICAL FORMULATION

$$h = W_0 \cdot x + \Delta_W \cdot x \implies h = W_0 \cdot x + (\alpha / r) * (B \cdot A) \cdot x$$

1. Base Weight W_0 : FROZEN during training (no gradient computation).

2. Matrix A ($d \times r$): Initialized with Gaussian distribution $N(0, \sigma^2)$.

3. Matrix B ($r \times k$): Initialized to EXACT ZERO so $\Delta_W = B \cdot A = 0$ at step 0.

4. Scaling Factor (α / r): Stabilizes training when rank r is modified.

```
# Lesson 28: PEFT LoRA Adapter Setup
from peft import LoraConfig, get_peft_model, TaskType

lora_config = LoraConfig(
    r=8,                                     # Rank dimension
    lora_alpha=16,                           # Scaling factor (multiplier = alpha/r = 2.0)
    lora_dropout=0.05,                       # Regularization dropout on LoRA layers
    task_type=TaskType.CAUSAL_LM,
    target_modules=['qkv_proj', 'o_proj'], # Target attention projections
    bias='none'
)
peft_model = get_peft_model(base_model, lora_config)
peft_model.print_trainable_parameters()
# Trainable params: 3,028,998 || All params: 3,824,108,544 || Trainable%: 0.0792% (>99.9% savings!)
```

Chapter 9: Quantization & QLoRA with SFTTrainer (Lesson 29)

Quantization compresses continuous 32-bit floating point parameters into lower precision discrete integer bins ($X_{\text{quant}} = \text{round}(X / \text{scale}) + \text{zero_point}$):

Precision	Bits / Param	VRAM (7B Model)	Quantization Mechanism
FP32 (Standard)	32 bits	~28.0 GB	Full single precision IEEE-754 floating point.
FP16 / BF16	16 bits	~14.0 GB	Half precision; BF16 preserves 8-bit dynamic range exponent.
INT8 (8-bit)	8 bits	~7.0 GB	Vector-wise quantization with outlier thresholding (BitsAndBytes).
NF4 (4-bit QLoRA)	4 bits	~3.8 GB	Information-theoretically optimal NormalFloat4 for Gaussian weights.

```
# Lesson 29: 8-Bit Quantized Loading & SFTTrainer with Sample Packing
from transformers import BitsAndBytesConfig, AutoModelForCausalLM
from trl import SFTConfig, SFTTrainer

# 1. Configure 8-bit quantization
bnb_config = BitsAndBytesConfig(load_in_8bit=True, llm_int8_threshold=6.0)
model = AutoModelForCausalLM.from_pretrained('gpt2-xl', quantization_config=bnb_config, device_map='auto')

# 2. Configure Supervised Fine-Tuning
sft_args = SFTConfig(
    output_dir='./qlora-alpaca',
    dataset_text_field='formatted_text',
    max_seq_length=512,
    packing=True, # Packs multiple short examples into 512 chunks (3-5x faster!)
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4, # Effective batch size = 16
    learning_rate=2e-4,
    logging_steps=10
)
trainer = SFTTrainer(model=model, args=sft_args, train_dataset=dataset, peft_config=lora_config, processing_class=tokenizer)
trainer.train()
```

Chapter 10: Local Production: Merging, GGUF & Ollama (Lessons 30–32)

Deploying fine-tuned models for local, low-latency edge inference involves fusing LoRA weights, converting into single-file binary GGUF format with 'llama.cpp', and packaging with Ollama:

```
# Step 1: Fuse LoRA adapter into base model weights
# W_merged = W_0 + (alpha / r) * B * A
merged_model = model.merge_and_unload()
merged_model.save_pretrained('./phi-3.5-mini-merged', safe_serialization=True)
tokenizer.save_pretrained('./phi-3.5-mini-merged')

# Step 2: Convert to GGUF binary format using llama.cpp
# python3 llama.cpp/convert_hf_to_gguf.py ./phi-3.5-mini-merged --outfile ./phi-3.5-f16.gguf --outtype f16

# Step 3: Ollama Modelfile Definition
# FROM ./phi-3.5-f16.gguf
# TEMPLATE """{{ if .System }}<|system|>
# {{ .System }}<|end|>
# {{ end }}{{ if .Prompt }}<|user|>
# {{ .Prompt }}<|end|>
# {{ end }}<|assistant|>
# """
# SYSTEM """You are an expert AI assistant specialized in instruction-following."""
# PARAMETER temperature 0.7
# PARAMETER stop "<|end|>"

# Step 4: Register and run in Ollama
# ollama create phi-3.5-custom -f Modelfile
# ollama run phi-3.5-custom
```

Chapter 11: Master Function, Class & Method Encyclopedia

Complete reference llama guide covering every critical class, function, method, and hyperparameter across the entire curriculum:

Function / Class	Library	Exact Purpose, Parameters & Best Practices
<code>Value(data)</code>	Custom Autograd	Scalar autograd node tracking computational graph operations (+, *, tanh) and computing backward derivatives.
<code>nn.Linear(in, out)</code>	PyTorch	Applies affine transformation $y = xW^T + b$. Initializes weights with Kaiming uniform by default.
<code>nn.BatchNorm1d(dim)</code>	PyTorch	Normalizes mini-batch activations to mean 0 and variance 1 with learnable scale γ and shift β .
<code>Conv1d(..., dilation)</code>	PyTorch	1D Convolution layer. Setting dilation > 1 expands receptive field exponentially for WaveNet sequence modeling.
<code>AutoTokenizer</code>	transformers	Instantiates the exact tokenizer architecture from model configuration (BPE, WordPiece, SentencePiece).

<code>apply_chat_template()</code>	transformers	Applies model's official Jinja template to role-based message lists (system/user/assistant).
<code>resize_token_embeddings()</code>	transformers	Resizes model vocabulary matrix when custom special tokens are registered to prevent index out of bounds.
<code>get_memory_footprint()</code>	transformers	Returns active VRAM/RAM consumed by model weights in bytes to verify quantization savings.
<code>DataCollatorWithPadding</code>	transformers	Pads mini-batch tensors dynamically to the maximum sequence length of that specific batch.
<code>DataCollatorForSeq2Seq</code>	transformers	Dynamically pads inputs and fills target label pad positions with -100 to ignore loss on padding tokens.
<code>LoraConfig</code>	peft	Defines low-rank decomposition rank r , scaling factor α , dropout, and target layers (qkv_proj).
<code>get_peft_model()</code>	peft	Injects trainable LoRA low-rank decomposition matrices while freezing all base model parameters.
<code>merge_and_unload()</code>	peft	Fuses trained LoRA delta weights permanently into base model weights: $W_{\text{new}} = W_0 + \frac{1}{r} \alpha BA$.
<code>BitsAndBytesConfig</code>	transformers	Enables 8-bit vector quantization (load_in_8bit) or 4-bit NormalFloat4 (load_in_4bit) loading.
<code>SFTTrainer</code>	trl	High-level Supervised Fine-Tuning trainer with support for sample packing, dataset text fields, and LoRA.
<code>convert_hf_to_gguf.py</code>	llama.cpp	Translates Hugging Face SafeTensors checkpoints into single-file binary GGUF format for CPU/Metal/GPU.
<code>ollama create / run</code>	Ollama CLI	Compiles a local containerized model from a Modelfile and executes real-time streaming inference.